

System Design Notes: Quorum Consensus

1. Introduction to Quorum Consensus

In distributed systems, data is often replicated across multiple nodes to ensure high availability and fault tolerance. However, replication introduces a challenge: **keeping the data consistent across all nodes**. **Quorum Consensus** is a protocol used to guarantee strong consistency in a distributed storage system (like Cassandra or DynamoDB) while still tolerating node failures. It defines the minimum number of nodes that must successfully acknowledge a read or write operation for that operation to be considered successful.

2. The Core Parameters (N , W , R)

Every quorum-based system relies on three configurable variables:

- **N (Replication Factor)**: The total number of nodes (replicas) that store a copy of the data.
- **W (Write Quorum)**: The minimum number of nodes that must acknowledge a **write** operation before it is considered successful.
- **R (Read Quorum)**: The minimum number of nodes that must respond to a **read** operation before the system returns the data to the client.

3. The Golden Rules of Consistency

Rule 1: Strong Consistency

To guarantee that a client will *always* read the most recently written data, the read and write quorums must overlap. This is achieved by ensuring:

$$W + R > N$$

If this condition is met, at least one node in the read quorum will have participated in the most recent write quorum, ensuring the latest data (usually resolved via timestamps) is returned.

Rule 2: Write Conflict Prevention (Split-Brain)

To prevent two different network partitions from accepting conflicting writes for the same data, the write quorum must represent an absolute majority of the nodes:

$$W > N / 2$$

4. Common Configurations & Trade-offs

Playing with **W** and **R** allows you to tune your system based on your application's specific needs (read-heavy vs. write-heavy).

A. Majority Quorum (The Balanced Approach)

- **Formula:** $W = \lfloor N/2 \rfloor + 1$, $R = \lfloor N/2 \rfloor + 1$
- **Example (N=5):** $W=3$, $R=3$.
- **Result:** $3 + 3 = 6$ ($6 > 5$). Strong consistency is guaranteed.
- **Use Case:** Standard databases where read and write latency are equally important, and fault tolerance is required.

B. Read-Heavy System (Fast Reads, Slow Writes)

- **Formula:** $R = 1$, $W = N$
- **Example (N=5):** $W=5$, $R=1$.
- **Result:** $5 + 1 = 6$ ($6 > 5$). Strong consistency guaranteed.
- **Trade-off:** Reads are lightning fast (only need to hit 1 node). However, writes are slow and fragile; if even *one* node is down, the system cannot accept writes (Available for reads, unavailable for writes).

C. Write-Heavy System (Fast Writes, Slow Reads)

- **Formula:** $W = 1$, $R = N$
- **Example (N=5):** $W=1$, $R=5$.
- **Result:** $1 + 5 = 6$ ($6 > 5$). Strong consistency guaranteed.
- **Trade-off:** Writes are incredibly fast, but reads are slow and fragile. If one node is down, reads fail. (Note: This violates the $W > N/2$ rule, meaning it's susceptible to concurrent write conflicts and usually requires complex vector clocks to resolve).

D. Eventual Consistency (High Availability)

- **Formula:** $W + R \leq N$
- **Example ($N=5$):** $W=2$, $R=2$.
- **Result:** $2 + 2 = 4$ ($4 \leq 5$).
- **Trade-off:** Very fast reads and writes, and highly tolerant of node failures. However, clients may read stale data (Eventual Consistency).

| 5. Summary of System Design Implications

1. **Latency:** Higher W or R means higher latency because the system must wait for the slowest node in the quorum to respond.
2. **Availability:** Higher W or R means lower availability. If $W=5$ and you only have 4 nodes online, your write fails.
3. **CAP Theorem:** Quorum tuning is how you navigate the CAP theorem in practice. You are trading Availability (A) for Consistency (C) based on how you set your W and R parameters relative to N .